

2.0.1 Kontrollstrukturen / Algorithmische Bausteine

Die Klasse Turtle ist ein bewegliches Dreieck, das seinen Weg wie mit einem Stift nachzeichnet.

Aufgabe 0 Predict

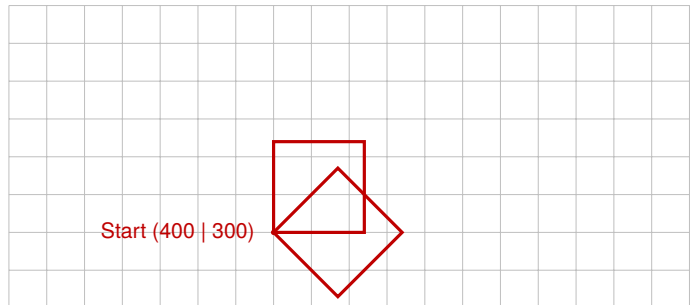
Zeichne in das Karofeld rechts, was der Java-Code auf den Bildschirm malt.

Hauptprogramm:

```
new World(800, 600);  
Zeichner z = new Zeichner(400, 300);  
  
z.quadrat(200);  
z.turn(-45);  
z.quadrat(200);
```

```
class Zeichner extends Turtle {  
  
    Zeichner(int x, int y) {  
        super(x, y, true);  
        setBorderColor(Color.aqua);  
        setBorderWidth(5);  
        penDown();  
    }  
  
    void quadrat(double seitenlaenge) {  
        for (int i = 0; i < 4; i++) {  
            forward(seitenlaenge);  
            turn(90); // gegen den Uhrzeigersinn  
        }  
    }  
}
```

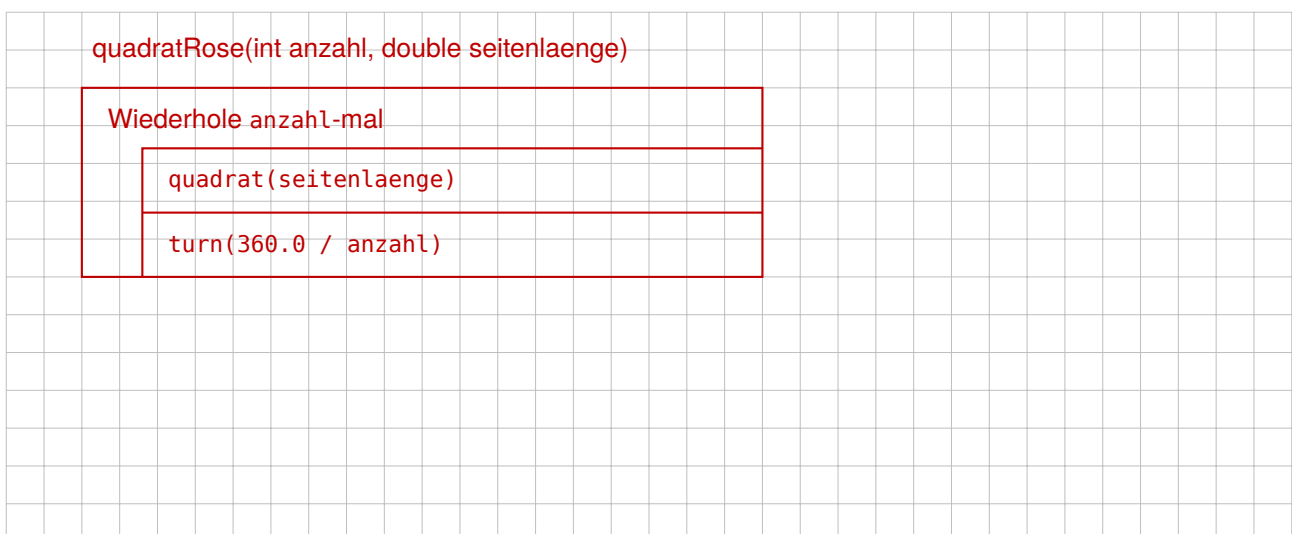
Meine Vorhersage:



Aufgabe 1 Projekt Quadratrosen

- a) Ausführen und Vorhersage prüfen. b) Vielecke. c) `quadratRose(anzahl, seitenlaenge)` schreiben.
d) Zeichne das Struktogramm deiner Methode. `quadrat(...)` ist darin nur ein Kästchen.

Name der Kontrollstruktur: Wiederholung mit fester Anzahl (for-Schleife)



Wiederholung mit fester Anzahl ↔ bedingte Wiederholung

Wiederholung mit fester Anzahl

```
for (int i = 0; i < 3; i = i + 1)
{
    erstelleQuadratAn(i, 1);
}
```

bedingte Wiederholung

```
int i = 0;
while (i < 3)
{
    erstelleQuadratAn(i, 1);
    i = i + 1;
}
```

Wandle die folgenden Wiederholungen jeweils in die andere Form um (in Java).

```
int i = 15;
while (i > 0) {
    println(i);
    i = i - 2;
}
```

```
for (int i = 15; i > 0; i = i - 2) {
    println(i);
}
```

```
for (int a = 5; a < 15; a++) {
    dingeTun();
}
```

```
int a = 5;
while (a < 15) {
    dingeTun();
    a++;
}
```

Aufgabe 2 Projekt CurvingFevers

- a)** Ausführen. **b)** Curver1 und Curver2 vergleichen. **c)** Steuerung ergänzen.
d) Übertrage deine Methode steuern() hierher und markiere: Bedingung, Sequenz bei wahr, Sequenz bei falsch (falls vorhanden). **e)** crashPruefen() ergänzen.

Name der Kontrollstruktur: bedingte Anweisung (hier einseitig)

Bedingung	
wahr	falsch
Sequenz 1	Sequenz 2

```
void steuern() {
    if (isKeyDown(Key.ArrowLeft)) { ← Bedingung
        turn(turningSpeed); ← Sequenz bei wahr
    } (kein sonst: einseitig)
    if (isKeyDown(Key.ArrowRight)) {
        turn(-turningSpeed);
    }
}
```

Bedingungen in Java

$x > y$	$x \leq y$	$x == y$	$x \neq y$	istWand()
größer	kleiner oder gleich	gleich	ungleich	Methode mit Rückgabe boolean

Verknüpfen: oder → || und → && Verneinen: nicht → !